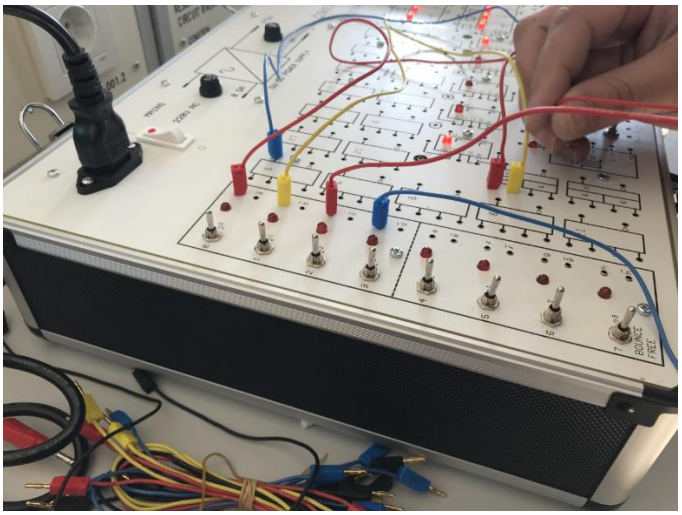
Sizinle, Mantık Devreleri üzerine geliştirdiğim ve laboratuvar ortamındaki fiziksel eğitim setlerine (Görsel1) alternatif oluşturmayı amaçlayan projem hakkında bilgi paylaşmak istiyorum.



Görsel1 1

Bu projeyi geliştirirken temel motivasyonum; mantık devreleri eğitimini donanımsal kısıtlamalardan kurtararak herkes için daha ulaşılabilir ve pratik hale getirmekti. Bu doğrultuda, tamamen Python tabanlı çalışan bir simülasyon ortamı hazırladım. Arından V2.0 sürümünde Tasarımı güçlendirip Web kısmına taşıdım.

Proje, fiziksel setlerin aksine, kullanıcının sınırsız sayıda mantık kapısı oluşturmasına ve bunları serbestçe test etmesine olanak tanımaktadır. Kodlama sürecine 3. sınıftayken (2024) başladım. Algoritmayı koda dökerken karşılaştığım teknik zorlukları aşarak, sistemi v1.2 sürümüyle kararlı ve çalışır hale getirmeyi başardım.

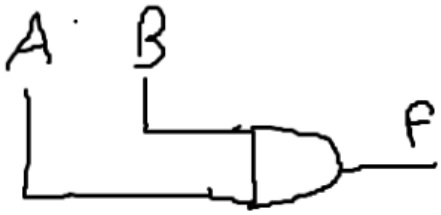
Projenin arayüzünü ve çalışma mantığını gösteren v1.2 sürümüne ait ekran görüntülerini

aşağıda/ekte bilgilerinize sunarım.

Sürüm v1.2 Çalışma mantığı şu şekilde:

Geliştirilen Python tabanlı simülasyon ortamında, görsel devre şemalarını algoritmik bir yapıya dönüştürmek amacıyla özgün bir tanımlama dili (sözdizimi) oluşturulmuştur. Bu yapıda temel prensip, her bir mantık kapısının işlemini Çıktı—KAPI=Girdi1,Girdi2 formatında tanımlamaktır.

Örneğin, basit bir VE (AND) kapısı işlemi şu şekilde ifade edilmektedir:

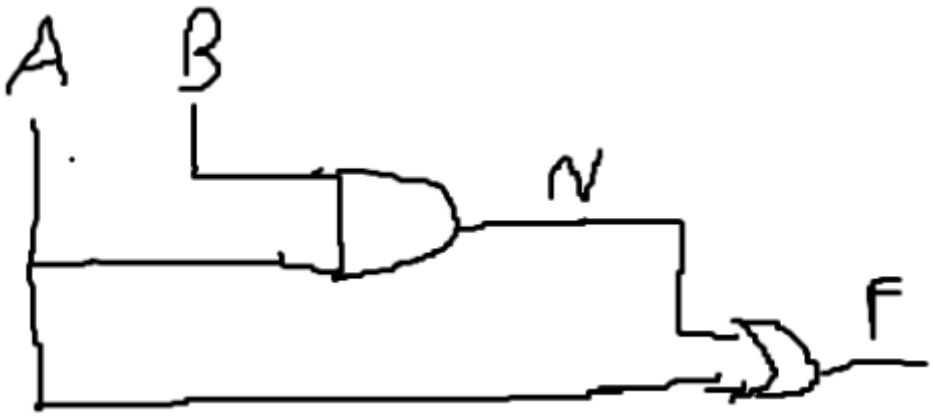


Görsel1 2

Görsel1.2 Devresini ele alalım. Bunu koda dökmek için . "***F—AND=A,B***" Şeklinde bir yapı oluşturdum. Bu ifade; **A** ve **B** giriş sinyallerinin AND işlemine tabi tutularak **F** çıktısının elde edildiğini belirtir.

Zincirleme İşlem Yeteneği ve Karmaşık Devre Tasarımı

Sistemin en kritik yeteneği, modüler işlem yapısıdır. Bir mantık kapısından elde edilen ara çıktı (intermediate output), sonraki aşamalarda başka bir kapının girdisi olarak tanımlanabilmektedir. Bu özellik, kullanıcının sınırsız sayıda kapı kullanarak karmaşık devre topolojileri oluşturmasına olanak tanır.



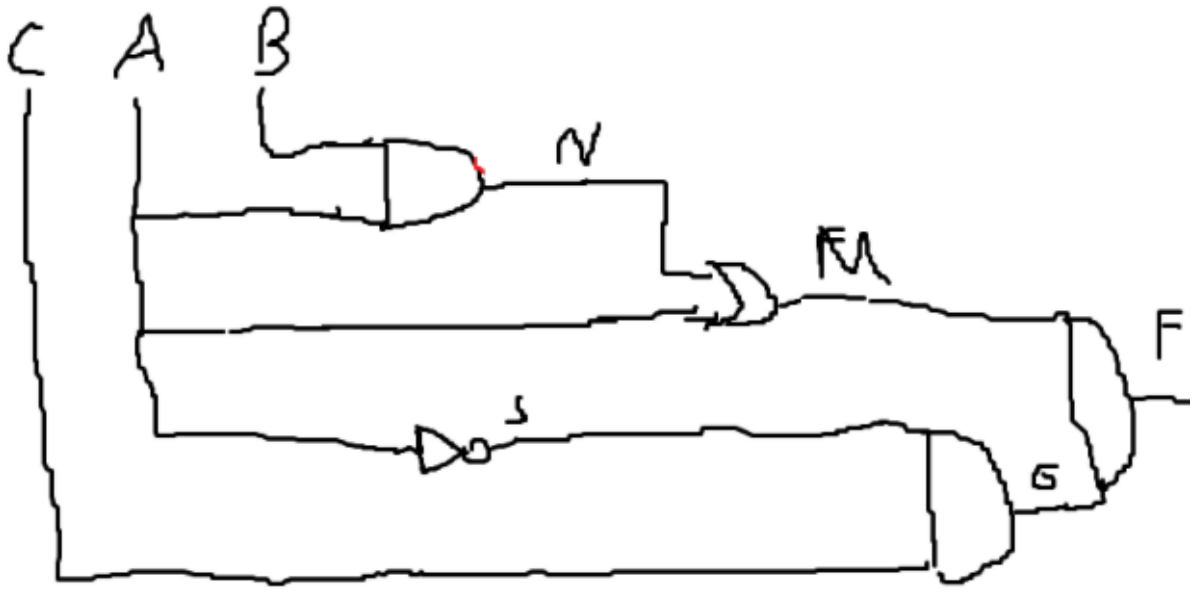
Görsel1 3

Bu durumu **Görsel 1.3** üzerinden örneklendirecek olursak:

1. **N—AND=A,B** : İlk adımda A ve B işlenerek **N** ara değeri elde edilir.

2. **F—OR=N,A** : Elde edilen N değeri, tekrar A girdisi ile OR işlemine sokularak **F** nihai sonucu bulunur.

Bu yöntemle, katmanlı mantık devreleri metin tabanlı olarak kolayca modellenebilmektedir.



Görsel1 4

Daha kapsamlı bir örnek olan **Görsel 1.4**'teki devre tasarımı ise şu adımlarla koda dökülmüştür:

- **N—AND=A,B** (A ve B'nin AND işlemi)
- **M—OR=N,A** (N ve A'nın OR işlemi)
- **S—DEGIL=A** (A'nın tersinin alınması - NOT işlemi)
- **G—AND=C,S** (C ve S'nin AND işlemi)
- **F—AND=M,G** (M ve G'nin AND işlemi ile Sonuç)

Simülasyon ve Doğruluk Değeri Analizi

Algoritma, tanımlanan bu komut setini ayrıştırarak devredeki bağımsız değişkenleri (Örn: **A, B, C**) bize sorar hangileri olduğunu. Kullanıcı tarafından bu değişkenlere ikili (binary)

doğruluk değerleri (0 veya 1) atandığında, sistem tüm ara işlemleri (N, M, S, G) simüle eder ve nihai **F** sonucunu hesaplar.

Aşağıda, Python simülasyonunda gerçekleştirilen işlem basamakları, kullanıcı etkileşimi ve çıktı analizleri sırasıyla sunulmuştur.

Devre Tanımlama ve Değişken Atama

Önceki bölümde açıklanan sözdizimi kullanılarak devre yapısı sisteme tanımlanmıştır (**Şekil 1.5**). Tanımlama işleminin ardından algoritma, devredeki mantık kapılarını analiz ederek kullanıcıdan gerekli giriş değişkenlerini talep etmektedir.

```
Kaç adet kod yazacaksın?
= 5
1. Satırı Yaz
> N--AND=A,B
2. Satırı Yaz
> M--OR=N,A
3. Satırı Yaz
> S--DEGIL=A
4. Satırı Yaz
> G--AND=C,S
5. Satırı Yaz
> F--AND=M,G
Kaç adet Girdi yazacaksın?
=
```

Görsel1.5

Sistemin beklediği giriş değişkenlerinin (A, B, C vb.) listelenmesi ve kullanıcı onayına sunulması **Şekil 1.6**'da gösterilmiştir. Bu aşama, karmaşık devrelerde kullanıcının hangi portlara veri gireceğini görmesi açısından önemlidir.

```
Kaç adet kod yazacaksın?
= 5
1. Satırı Yaz
> N--AND=A,B
2. Satırı Yaz
> M--OR=N,A
3. Satırı Yaz
> S--DEGIL=A
4. Satırı Yaz
> G--AND=C,S
5. Satırı Yaz
> F--AND=M,G
Kaç adet Girdi yazacaksın?
= 3
Girdi ne? [Orn: A,B..]
> C
Girdi ne? [Orn: A,B..]
> A
Girdi ne? [Orn: A,B..]
> B
```

Görsel1.6

Simülasyonun Yürütülmesi ve Sonuç

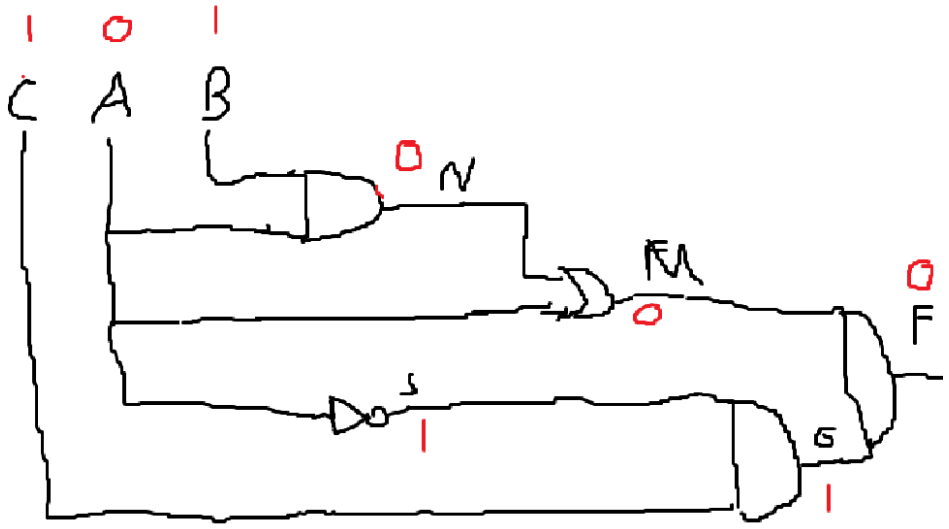
Giriş değişkenleri belirlendikten sonra, her bir değişkene ikili (0 veya 1) doğruluk değerleri atanır. Simülasyon motoru, bu değerleri mantık kapıları zinciri boyunca işleyerek nihai **F** sonucunu hesaplar. **Şekil 1.7**'de görüleceği üzere, girilen değerler doğrultusunda sistem çıkışı **F=0** olarak bulunmuştur.

```
C için değer girin (0 veya 1): 1
A için değer girin (0 veya 1): 0
B için değer girin (0 veya 1): 1
0
0
1
1
0
Sonuç: F = 0
Yeni değer girmek istiyor musunuz? [1: Evet, 0: Hayır]
=
```

Görsel1.7

Kullanıcı Deneyimi ve Test Sürekliliği

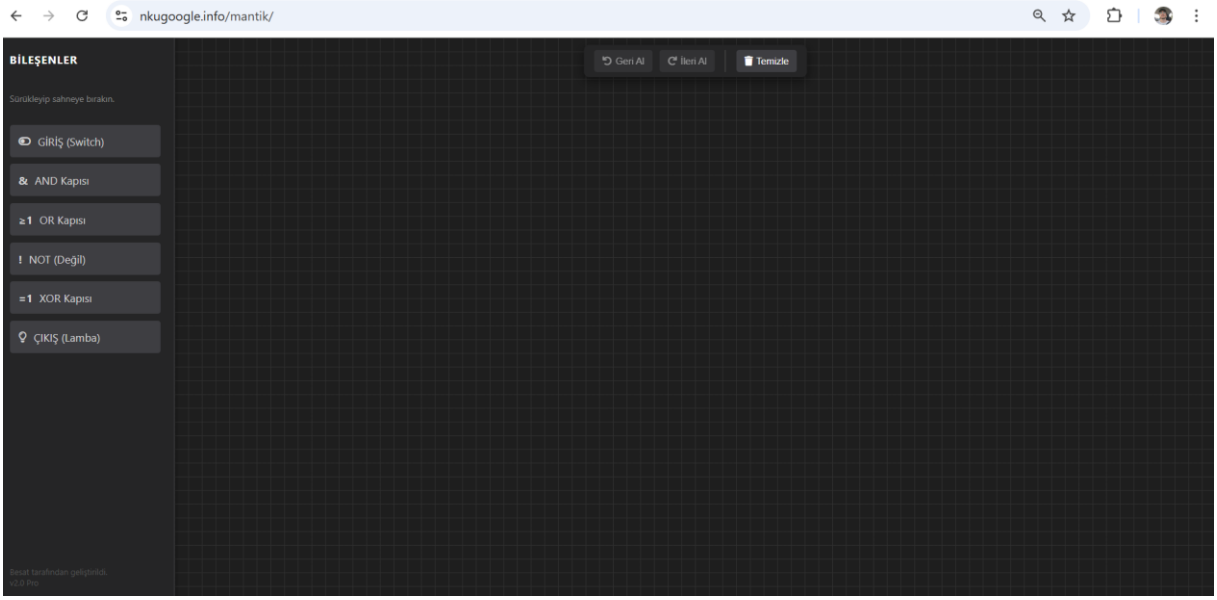
Analiz sürecinde kullanıcı deneyimini (UX) iyileştirmek adına, "Sürekli Test Döngüsü" geliştirilmiştir. Kullanıcı bir hesaplama yaptıktan sonra, devreyi baştan tanımlamasına gerek kalmadan, sadece giriş değerlerini değiştirerek farklı senaryoları test edebilir. **Şekil 1.7**, sistemin farklı girişlerle yeniden test edilmesini ve doğrulama sürecini göstermektedir.



Görsel1.8

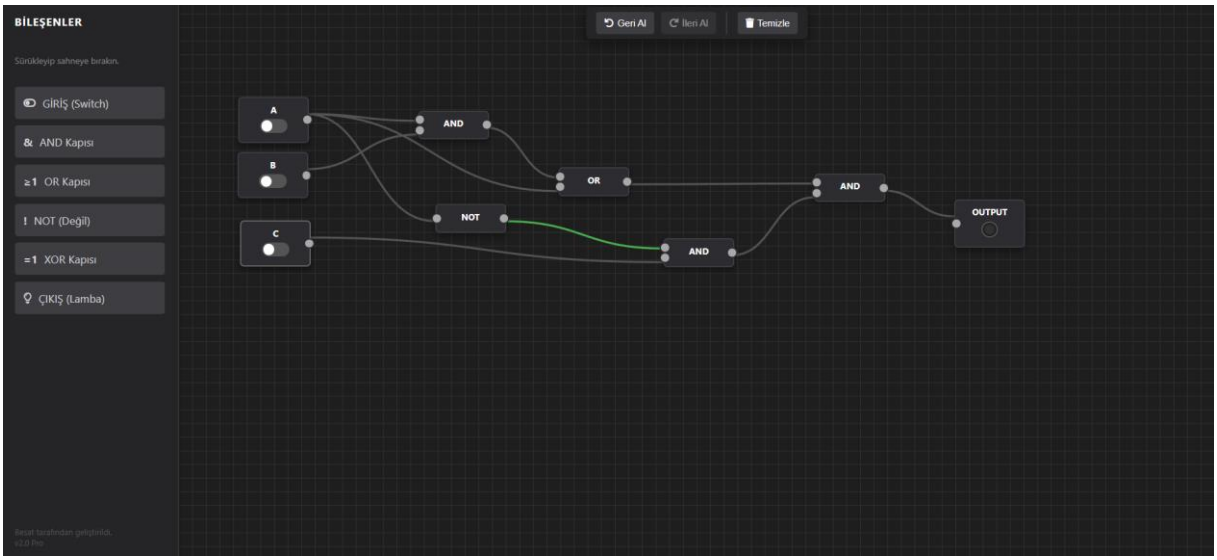
Şuan Aktif olarak yürütülen: Web Tabanlı v2.0 Sürümü

Mevcut Python sürümü (v1.2) komut satırı üzerinden çalışmakta olup, projenin bir sonraki aşamasında (v2.0) erişilebilirliği artırmak hedeflenmekteydi. Bu kapsamda, sürükle-bırak (drag-and-drop) yöntemiyle çalışan, görsel arayüze sahip web tabanlı bir simülasyon ortamının geliştirilmesi üzerine çalışmalar devam etmektedir. Şuan olan güncel sürüm Görsel1.9 da gözükmektedir. Sol taraftan sürükle bırak yaparak Kapıları bırakıyoruz.



Görsel1.9

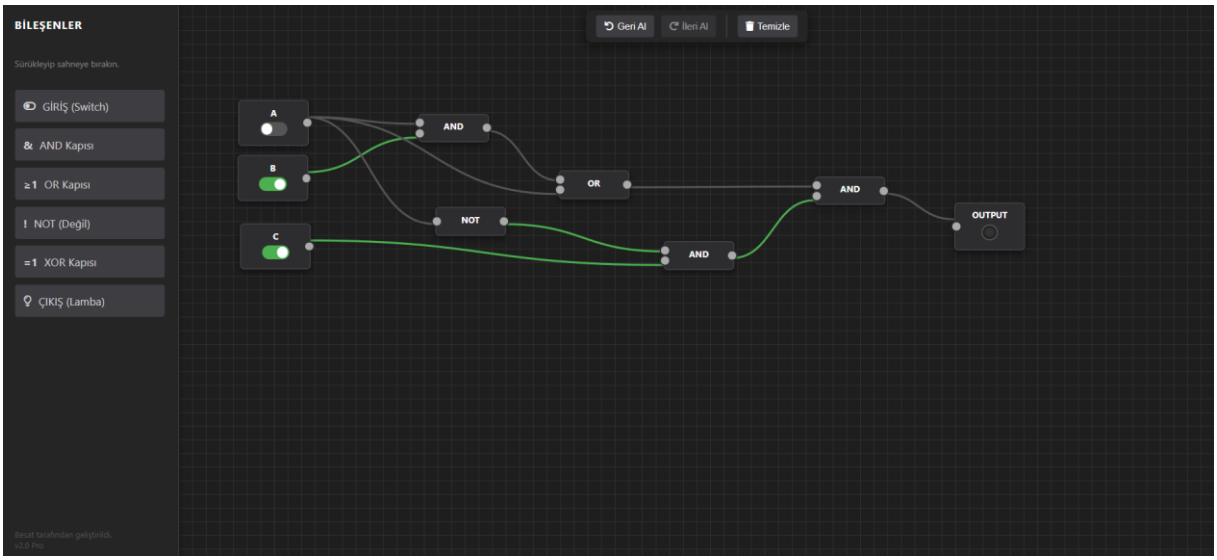
Görsel2 de gözüktüğü üzere sürükle bırak yaptıktan sonra kapı giriş ve çıkışlarını beyaz noktaya basarak (basılı tutarak) sürükleyip diğer çıkış ile birleştiriyoruz.



Görsel2

Geliştirilen web tabanlı (v2.0) sürümde, kullanıcıların mantık akışını daha rahat takip edebilmeleri için renk kodlu bir geri bildirim mekanizması kullanılmıştır. Arayüzde **"Yeşil"** renk mantıksal **'1'** (Yüksek/High), **"Siyah"** (veya sönük) renk ise mantıksal **'0'** (Düşük/Low) seviyesini temsil etmektedir.

Önceki bölümde (v1.2) Python kodlarıyla doğrulanan devre yapısı, bu kez web arayüzünde modellenmiş ve aynı test koşulları uygulanmıştır. Giriş değerleri, önceki deneyle tutarlı olması adına **A=0, B=1, C=1** olarak ayarlanmıştır.

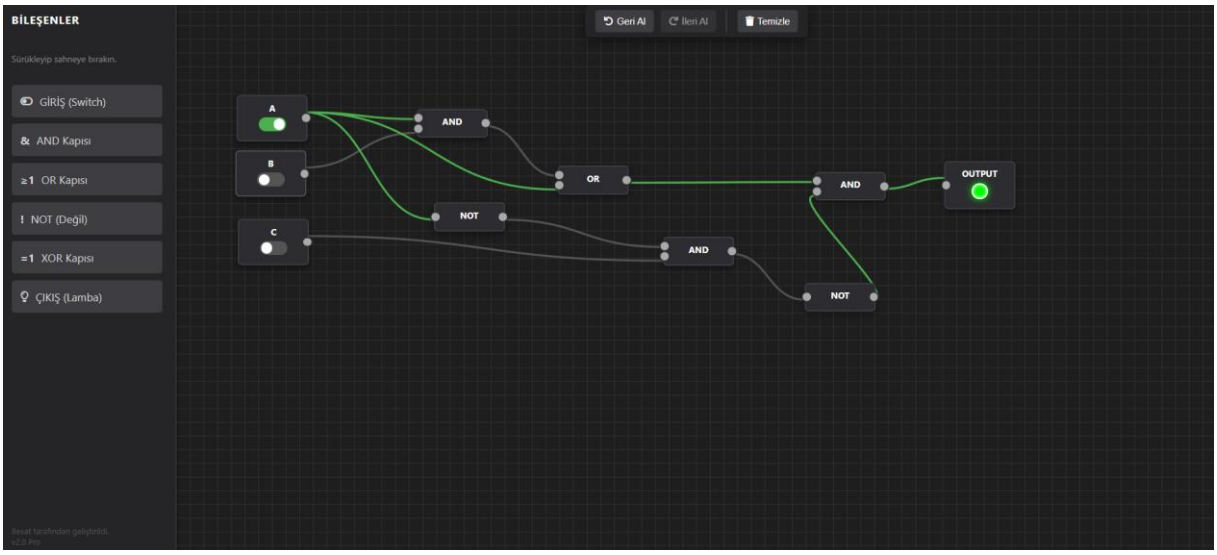


Görsel2.1

Şekil 2.1'deki simülasyon sonucu incelendiğinde; nihai çıkışın aktif olmadığı (Siyah/0) görülmektedir yani çalışıyor sorunsuz. Girişler ile oynayarak kablolarda 1-0 dolaşımı takip edebilir ve lambanın yanıp yanmama durumlarına bakabiliriz. Bu devre örneğine göre hep “0” Oluyor ve lamba yanmıyor. Sistemin en güçlü yönü olan "görsel sinyal takibi" sayesinde, devrenin ara katmanlarında hangi hatlardan '1' (Yeşil), hangi hatlardan '0' (Siyah) geçtiği anlık olarak gözlemlenebilmektedir. Bu özellik, hatalı tasarımların tespitini (debug) büyük ölçüde kolaylaştırmaktadır.

Dinamik Devre Modifikasyonu

Mevcut devre yapısında çıkışı '1' elde etmeye çalışırsak edemeyiz (her olasılığı denedim) burada geliştirdiğim yazılım bir avantajı daha doğuyor ekleme yapabiliyorum. Bu yüzden “1” elde edilememesi üzerine, sistemin esnekliği test edilerek devreye müdahale edilmiştir. "Sürükle-Bırak" (Drag-and-Drop) özelliği kullanılarak devreye anlık olarak yeni mantık kapıları eklenebilmektedir. Bu senaryoda, çıkışı mantıksal '1' seviyesine çekmek amacıyla devre sonuna bir **DEĞİL (NOT)** kapısı entegre edilmiştir.



Görsel2.2

Yapılan bu anlık modifikasyon sonucunda, **Şekil 2.2**'de görüldüğü üzere sinyal terslenmiş ve devre çıkışı aktif (Yeşil/1) hale gelmiştir.

Bitiş Yazısı (Sonuç Bölümü)

SONUÇ VE DEĞERLENDİRME

Bu çalışmada, mantık devreleri eğitimi ve tasarımı için geliştirilen, erişilebilirliği yüksek bir simülasyon ekosistemi sunulmuştur. Proje, donanımsal kısıtlamaları ortadan kaldıran Python tabanlı sözdizimi mimarisi (v1.2) ile başlamış ve kullanıcı deneyimini merkeze alan web tabanlı görsel arayüz (v2.0) ile evrilmiştir.

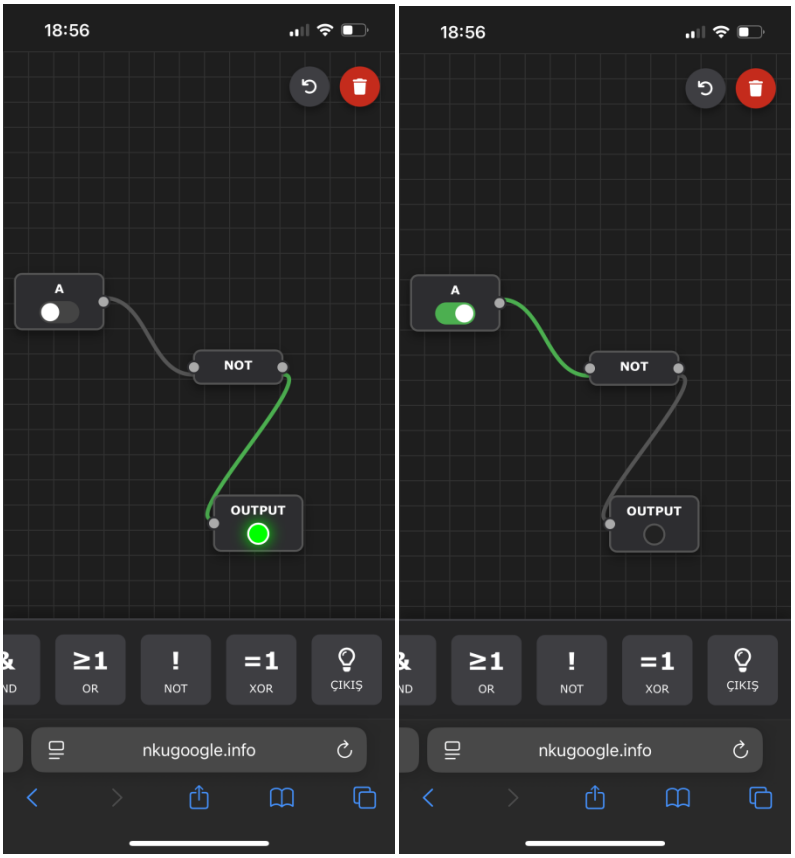
Geliştirilen sistemin temel kazanımları şunlardır:

1. **Sınırsız Tasarım Özgürlüğü:** Fiziksel deney setlerindeki kapı sayısı sınırını ortadan kaldırarak karmaşık devrelerin tasarlanmasına olanak tanımıştır.
2. **Görselleştirme:** Sinyal akışının renk kodlarıyla (Yeşil/Siyah) takibi, soyut mantık işlemlerinin öğrenciler ve meraklılar tarafından daha somut bir şekilde kavranmasını sağlamıştır.
3. **Etkileşim:** Sürükle-bırak yapısı ve anlık test imkanı, deneme-yanılma yoluyla öğrenmeyi teşvik etmektedir.

Görsel1 cihazı gibi fiziksel setlerin sağladığı temeli dijital dünyaya taşıyan bu proje, mantık devreleri eğitimini daha demokratik, ulaşılabilir ve anlaşılır kılma yolunda önemli bir adım

teşkil etmektedir. Gelecek çalışmalarda, sistemin daha karmaşık entegre devreleri destekleyecek şekilde genişletilmesi ve mobil platformlara uyarlanması hedeflenmektedir.

Sitemi buraya bırakıyorum buradan girip deneyebilirsiniz:

https://nkugoogle.info/mantik/

Mobil Tasarım örnek görüntüsü: